CSC311: Introduction to Machine Learning

Project Proposal

Which Sport Is It? Predicting a Sport from Survey Answers

Submitted by

Group 62629

Junde Yang, Sixuan Song, Yuzhou Wu, Zhihan Fan

University of Toronto
Fall 2026

1 Data Exploration

1.1 Test set: set aside before any exploration

- **Split by respondent, before looking at any feature.** The 870 rows come from 290 respondents, each answering once per sport (*Baseball*, *Basketball*, *Soccer*). With a fixed seed (`split_data.py`), 202 / 44 / 44 respondents give 606 / 132 / 132 rows for training / validation / test, each exactly one third per class (no class imbalance). Keeping a person's answers together stops a model from exploiting someone's writing style, which would not transfer to the TA test set. The ID is an arbitrary index and is never a feature.
- **Everything below uses the 606 training rows only** (`explore.py` never reads the other files); the test file will be opened once, for the final estimate.

1.2 Feature types and distributions

Table 1: Feature types, missing rate (training split) and encoding.

Feature	Type	Missing	Encoding
Ease of starting (0–10)	numeric	0%	median-impute, standardise
Strategy, Injury, Individual	Likert (5 levels)	0%	ordinal 1–5, standardise
Where, Season, Music	multi-select	0%	multi-hot (7, 4, 10 indicators)
Food, Colour, Song, Feeling	free text	2.0, 0.8, 7.8, 0.0%	binary bag-of-words
Price willing to pay	free text → $	0.7% (+2.3% no number)	cap, $\log(1+x)$, standardise

- **Ease** is roughly symmetric (mean 5.4, SD 2.4). **Likert** items lean towards *Agree*, with pairwise correlations ≤ 0.31, so none is redundant. **Price** is very right-skewed (skewness 6.7, median \$100, max \$5000, 13% at \$0).
- **Free text** is short (median 1–2 words), so the encoding is sparse: 581 features for 606 rows; 417 of 564 binary features occur in <1% of rows. **Near-constant:** *Outdoor* (87–91% in every class) and *Summer* (89–93%) carry little information.

1.3 Missing values, outliers and inconsistencies

Table 2 lists each issue found in the training split and how we handle it.

Table 2: Data-quality issues (training split) and their handling.

Issue	Example (frequency)	Handling
Several forms of "empty"	NaN, "", "n/a", "-"	One missing marker plus a 0/1 *missing* indicator per question, since missingness is informative (*Song*: 10.9 / 9.9 / 2.5% by class); training-median imputation.
"none"	"none"	A real answer, kept as a word.
Text variants	"Hotdog", "hot dogs", "[this sport]" (staff redaction [9])	Lower-case; drop placeholders and stop words; join "hot dog"; strip plural "s".
Price formats	"\$1,000", "200CAD", "100-200", "Nothing", "very willing"	Parse the number; midpoint of ranges; refusals → \$0; no number (2.3%) → missing.
Price outliers	IQR rule flags 12% (above \$425)	Too many to be errors: cap at \$1000 (8 answers), then $\log(1+x)$; skewness 6.7 → −1.1.

1.4 Preprocessing and why

- **Likert → ordinal 1–5**, not one-hot: one feature keeps the order instead of five, which matters when $p \approx n$. **Multi-select → multi-hot** 0/1, since several options can be chosen.
- **Numeric → standardised** with the training mean and SD [4, Sec. 7], because gradient-based training, L2 penalties and neural networks are scale-sensitive.
- **Free text → binary bag-of-words**, one vocabulary per question; presence, not counts, since answers are 1–2 words. Words seen `<min_df` times are dropped and unseen words ignored; `min_df` and binary vs. TF-IDF will be tuned (Section 2).
- **No leakage:** every learned quantity (medians, means, SDs, vocabularies) is fitted on training data only (`preprocess.py`), to avoid data leakage [12].

1.5 Which features are predictive

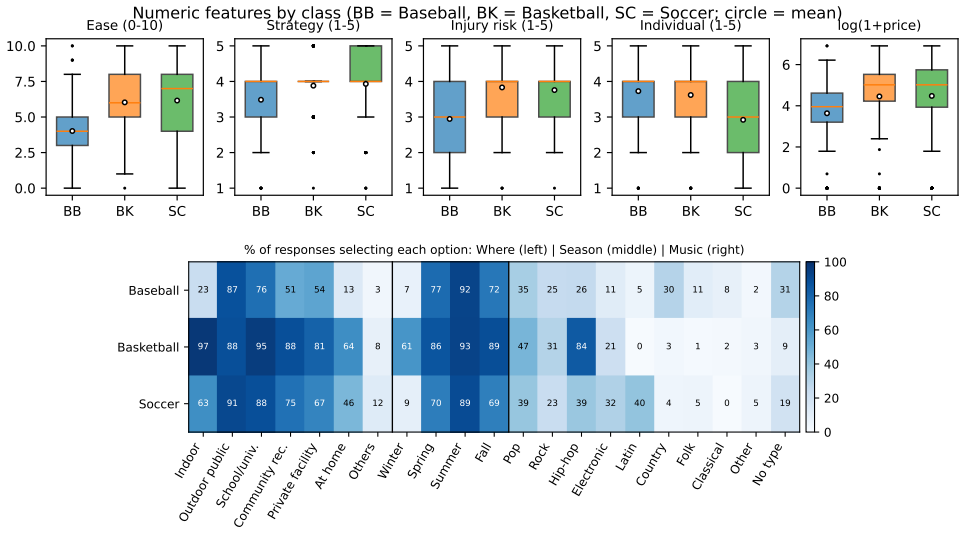


Figure 1: Training split only. **Top:** numeric features; Baseball rates *ease* and *injury* lower, *strategy* and *price* barely differ. **Bottom:** % of each class selecting each option; *Indoor*, *Winter*, *Hip-hop* point to Basketball, *Latin* to Soccer, *Country* to Baseball.

- **Numeric and Likert** (Fig. 1, top; gap/SD = gap between class means over within-class SD). *Ease*: Baseball 4.0 vs. Basketball/Soccer 6.0/6.2 (gap/SD = 1.0). *Injury risk*: Baseball 2.95 vs. 3.8. *Individual brilliance*: Soccer 2.9 vs. 3.6. *Strategy* and *price* are weak (gap/SD < 0.5).
- **Multi-select** (Fig. 1, bottom). *Indoor* 23 / 97 / 63% (Baseball / Basketball / Soccer); *Winter* 61% for Basketball, under 10% otherwise; *Hip-hop* 84% for Basketball; *Latin* 40% for Soccer; *Country* 30% for Baseball.
- **Free text:** the strongest features are words used almost only for one class: “waka” and “shakira” (100% Soccer), “take me out to the ball game” (≥95% Baseball), “green” (95% Soccer), “orange” (95% Basketball). The features with the highest mutual information with the label are `song:waka`, `where=Indoor` and `colour:green`.
- **Implications.** Many sparse indicators, each adding evidence for one class → regularised softmax regression; thresholds (low *ease*) and interactions (*Indoor* and *Winter*) → random forest, MLP; $p \approx n$ (581 vs. 606) → strong regularisation for all.

2 Planned Methodology

Model families. Each family is motivated by Section 1, and each can be re-implemented with numpy alone in `pred.py`.

1. **Softmax (multinomial logistic) regression with L2.** Most features are sparse indicators, and each adds a small, roughly independent amount of evidence for one class. That is exactly the additive log-odds model that softmax regression assumes. It has the lowest variance of our candidates, which matters with 581 features and 606 rows, and its weights are interpretable.
2. **Random forest** [1], [3, Sec. 6]. Several effects are threshold-shaped (Baseball if ease ≤ 4, Basketball if *Winter*) and may interact. Trees capture both without scaling and are unaffected by the remaining skew. Averaging many decorrelated trees removes most of a single tree's variance.
3. **Multilayer perceptron (MLP)** [6]. Hidden units can learn non-linear combinations of features (e.g. a word that matters only together with a season). With L2 regularisation and early stopping, it is a flexible middle ground between the two models above.

We decided not to use k-nearest neighbours. Our encoding has 581 features for 606 rows, and most of them are rare word indicators (417 occur in under 1% of rows), so distances would be dominated by many uninformative words rather than the few strong features found in Section 1 (the curse of dimensionality [4, Sec. 8]). kNN weights every feature equally and cannot learn which ones matter. We also ruled out fine-tuning a language model, because 606 rows are too few and `pred.py` may only use numpy and pandas [2, Sec. 3.4].

Training setup. Let $\mathbf{x}^{(i)} \in \mathbb{R}^D$ be an encoded response with label $y^{(i)} \in \{1, 2, 3\}$. Models will be trained with scikit-learn 1.9 [11] on top of our own `preprocess.py`, and every random seed is fixed to 311.

- *Softmax regression* [5, Secs. 3.3, 4.1]: we will minimise the mean cross-entropy plus an L2 penalty [7, Sec. 2], $\frac{1}{N}\sum_i -\log p(y^{(i)} \mid \mathbf{x}^{(i)}) + \frac{\lambda}{2}\|\mathbf{W}\|_F^2$, where $C = 1/(\lambda N)$. The objective is convex, so we will use L-BFGS (tolerance 10^{-4}, at most 2000 iterations), and every run should reach the same optimum.
- *Random forest:* Gini impurity and bootstrap samples. Trees are grown greedily, so there is no gradient optimiser. Regularisation comes from `max_depth`, `min_samples_leaf` and `max_features`.
- *MLP:* ReLU hidden layers, a softmax output and the cross-entropy loss. We will use Adam [10] ($\beta_1 = 0.9$, $\beta_2 = 0.999$) with mini-batches [8, Sec. 2.3] of 32, a constant learning rate η and an L2 penalty α. Early stopping will be done by tuning the number of epochs with grouped CV. We will not use scikit-learn's built-in early stopping, because its internal validation split is made by row, not by respondent [14], and can put the same respondent on both sides.

Validation scheme. (1) *Tuning:* 5-fold **grouped** cross-validation (`GroupKFold` [13] on respondent ID) on the 606 training rows. The `FeatureBuilder` will be re-fitted inside every

fold, so the vocabulary, medians and μ, σ never see the held-out fold. (2) *Model selection:* each family will be refit on all training rows with its best hyperparameters and scored once on the 132 validation rows, and the highest validation macro-F1 wins. (3) *Final estimate:* the chosen model will be trained on training + validation and scored once on the 132 test rows, which gives our single-number test estimate. Only after that will we retrain on all 870 rows for `pred.py`.

Hyperparameter search (Table 3). Scale parameters (C, α, η) will be searched on log-spaced values. Each range is chosen to span from likely underfitting ($C = 10^{-3}$, depth 3) to likely overfitting ($C = 10^{2}$, unlimited depth). If the best value is at the edge of a range, we will extend the range and search again. Softmax regression will use a full grid of 88 settings, and the forest and the MLP will each use 60 random-search settings, so all three families get a similar budget. Settings will be compared by mean grouped-CV macro-F1, with accuracy as the tie-breaker. If two settings are within one standard error, we will pick the simpler one. To show that the ranges are reasonable, we will plot validation curves (CV score against C, `max_depth` and α) with the peak inside each range. **All three families will be fully tuned before they are compared.** Tuning only the winner would be unfair to the others.

Table 3: Planned hyperparameter search (grouped 5-fold CV on the training split). The two text settings in the first row are searched together with each model's own hyperparameters, e.g. the softmax grid has $11 \times 4 \times 2 = 88$ settings.

Model	Search	Hyperparameter	Planned values
All three	—	`min_df`	$\{1, 2, 3, 5\}$
		text weighting	{binary, TF-IDF}
Softmax regression	full grid (88)	$C = 1/(\lambda N)$	11 log-spaced values, 10^{-3} to 10^{2}
Random forest	random (60)	`n_estimators`	$\{100, 300, 600\}$
		`max_depth`	$\{3, 5, 10, 20, \text{None}\}$
		`min_samples_leaf`	$\{1, 2, 5, 10\}$
		`max_features`	$\{\sqrt{D},\ \log_2 D,\ 0.1D,\ 0.3D\}$
MLP	random (60), 3 seeds each	hidden layers	$(32), (64), (128), (64, 32)$
		L2 penalty α	log-uniform on $[10^{-5}, 1]$
		learning rate η	$\{10^{-4},\ 3{\times}10^{-4},\ 10^{-3},\ 3{\times}10^{-3}\}$
		epochs (early stopping)	$\{25, 50, 100, 200, 400\}$

Evaluation metrics. For class k, $\mathrm{P}_k = \frac{\mathrm{TP}_k}{\mathrm{TP}_k + \mathrm{FP}_k}$, $\mathrm{R}_k = \frac{\mathrm{TP}_k}{\mathrm{TP}_k + \mathrm{FN}_k}$ and $\mathrm{F1}_k = \frac{2\mathrm{P}_k\mathrm{R}_k}{\mathrm{P}_k + \mathrm{R}_k}$.

- **Accuracy:** `pred.py` is graded on test accuracy [2, Sec. 3.4], and our classes are balanced, so it is a meaningful headline number.
- **Macro-F1** $= \frac{1}{3}\sum_k \mathrm{F1}_k$ (the selection metric) weights every sport equally and drops sharply when a model neglects one class. For example, always predicting one class gives 33% accuracy but only 0.17 macro-F1. The TA-written test set may differ from ours in class mix and wording, so we want a model that does well on all three sports. We prefer macro-over weighted-averaging because no sport matters more than another.

- **Per-class precision/recall and the confusion matrix** show which sports get confused. We expect Basketball vs. Soccer, whose *ease* and *injury* distributions overlap (Fig. 1).
- **Mean ± SD over the 5 folds**, so that we do not over-interpret differences smaller than the fold-to-fold noise.

Planned additional analyses. (i) A *feature-group ablation* that drops song, colour, food, feeling or the multi-select group in turn. TAs may name different songs, so we want to know how much we rely on a few strong words. (ii) A *soft-voting ensemble* of the three tuned models, to be evaluated with the same protocol. (iii) One-hot vs. ordinal Likert encoding, and word bigrams, both to be tested inside the same CV loop.

References

[1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[2] Alice Gao. CSC311 Machine Learning Project Instructions, Fall 2026 (v1.0). https://q.utoronto.ca/courses/441683/modules/items/8158748, 2026.

[3] Alice Gao et al. CSC311 course notes: Ensemble methods. https://www.teach.cs.toronto.edu/~csc311notes/bias_variance_decomposition/ensemble.html, 2026. University of Toronto, accessed October 2026.

[4] Alice Gao et al. CSC311 course notes: k-nearest neighbours. https://www.teach.cs.toronto.edu/~csc311notes/supervised_learning/knn.html, 2026. University of Toronto, accessed October 2026.

[5] Alice Gao et al. CSC311 course notes: Multi-class classification. https://www.teach.cs.toronto.edu/~csc311notes/linear_classification/softmax_regression.html, 2026. University of Toronto, accessed October 2026.

[6] Alice Gao et al. CSC311 course notes: Multi-layer perceptron. https://www.teach.cs.toronto.edu/~csc311notes/neural_networks/multi_layer_perceptron.html, 2026. University of Toronto, accessed October 2026.

[7] Alice Gao et al. CSC311 course notes: Regularization. https://www.teach.cs.toronto.edu/~csc311notes/linear_regression/regularization.html, 2026. University of Toronto, accessed October 2026.

[8] Alice Gao et al. CSC311 course notes: Stochastic gradient descent. https://www.teach.cs.toronto.edu/~csc311notes/linear_classification/sgd.html, 2026. University of Toronto, accessed October 2026.

[9] Alice Gao and Marina Tawfik. CSC311 Project Updates. https://q.utoronto.ca/courses/441683/discussion_topics/3637335, 2026. Quercus announcement, October 9, 2026.

[10] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[11] Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[12] scikit-learn developers. Common pitfalls and recommended practices: Data leakage. https://scikit-learn.org/stable/common_pitfalls.html, 2026. Accessed October 2026.

[13] scikit-learn developers. GroupKFold. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupKFold.html, 2026. Accessed October 2026.

[14] scikit-learn developers. MLPClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html, 2026. Parameter early_stopping; accessed October 2026.